PHP: Sintaxis, tipos de datos, variables, constantes, operadores, conversión de datos, include y require

1. Introducción

PHP (PHP: Hypertext Preprocessor) es un lenguaje de programación de código abierto diseñado principalmente para el desarrollo de aplicaciones y páginas web dinámicas. Fue creado para facilitar la generación de contenido web desde el servidor y, con el paso del tiempo, se ha convertido en un lenguaje ampliamente utilizado para desarrollar sitios web y aplicaciones.

PHP es un lenguaje que se ejecuta principalmente **en el servidor**. Esto significa que el navegador del usuario no recibe directamente el código PHP, sino el resultado que genera dicho código.

Por ejemplo, si tenemos:

```
<?php

echo "Hola, mundo";
```

El servidor ejecuta PHP y el navegador recibe el resultado:

```
Hola, mundo
```

Una de las principales ventajas de PHP es que puede combinarse con tecnologías como **HTML, CSS y JavaScript**, además de permitir trabajar con bases de datos.

PHP también permite:

- Crear páginas web dinámicas.
- Procesar formularios.
- Gestionar sesiones y usuarios.
- Trabajar con bases de datos.
- Crear sistemas de autenticación.
- Manipular archivos.
- Crear APIs y servicios web.
- Organizar aplicaciones mediante funciones, clases y diferentes archivos.

Para aprender PHP es importante conocer primero sus elementos básicos: la sintaxis, los tipos de datos, las variables, las constantes, los operadores, la conversión de datos y la inclusión de archivos mediante `include` y `require`.

---

2. Sintaxis de PHP

La **sintaxis** es el conjunto de reglas que debemos seguir para escribir correctamente un programa en PHP.

Al igual que ocurre con otros lenguajes de programación, si no respetamos las reglas de sintaxis podemos obtener errores y el programa puede no ejecutarse correctamente.

2.1. Etiquetas de apertura

El código PHP comienza normalmente con:

```
<?php
```

Por ejemplo:

```
<?php

echo "Hola, mundo";
```

En un documento que combine PHP y HTML podemos utilizar:

```
<!DOCTYPE html>
<html>
<head>
    <title>Mi página</title>
</head>
<body>

<?php
echo "Hola desde PHP";
?>

</body>
</html>
```

El servidor interpreta únicamente la parte correspondiente a PHP.

2.2. Etiqueta de cierre

PHP permite utilizar una etiqueta de cierre:

```
?>
```

Por ejemplo:

```
<?php

echo "Hola";
```

```
?>
```

Sin embargo, cuando el archivo contiene exclusivamente PHP, normalmente se recomienda no utilizar la etiqueta de cierre:

```
<?php

echo "Hola";
```

Esto ayuda a evitar que se introduzcan accidentalmente espacios o caracteres después del código PHP.

---

3. Instrucciones y punto y coma

Las instrucciones PHP normalmente terminan con `;`.

Ejemplo:

```
<?php

$nombre = "Pedro";
$edad = 25;

echo $nombre;
echo $edad;
```

Cada instrucción está separada por un punto y coma.

Si escribimos:

```
$nombre = "Pedro"
echo $nombre;
```

podemos obtener un error de sintaxis porque falta `;` después de `"Pedro"`.

---

4. Comentarios en PHP

Los comentarios permiten escribir explicaciones dentro del código sin que PHP las ejecute.

Son especialmente importantes cuando trabajamos en proyectos grandes.

4.1. Comentarios de una línea

Podemos utilizar `//`:

```
// Guardamos el nombre del usuario
$nombre = "Ana";
```

También podemos utilizar `#`:

```
# Guardamos la edad
$edad = 20;
```

Actualmente `//` es una forma muy habitual de escribir comentarios de una línea.

4.2. Comentarios de varias líneas

Para comentarios que ocupan varias líneas utilizamos:

```
/*
Este comentario puede ocupar
varias líneas.
*/
```

Por ejemplo:

```
/*
Calculamos el precio final
añadiendo el IVA.
*/

$total = $precio + $iva;
```

Los comentarios no afectan al funcionamiento del programa.

---

5. Mostrar información en PHP

Una de las instrucciones más sencillas para mostrar información es `echo`.

Ejemplo:

```
<?php

echo "Hola, mundo";
```

También podemos mostrar variables:

```
$nombre = "Laura";

echo $nombre;
```

Podemos combinar texto y variables:

```
$nombre = "Laura";
$edad = 20;

echo "Mi nombre es $nombre y tengo $edad años.";
```

Otra función útil es `var_dump()`, especialmente durante el desarrollo, porque muestra el tipo y el valor de una variable:

```
$edad = 20;

var_dump($edad);
```

El resultado será similar a:

```
int(20)
```

---

6. Tipos de datos en PHP

Los tipos de datos indican qué clase de información estamos almacenando.

Los principales tipos que debemos conocer son:

Tipo	Descripción	Ejemplo
`string`	Texto	`"Hola"`
`int`	Número entero	`25`
`float`	Número decimal	`19.95`
`bool`	Verdadero o falso	`true`
`array`	Conjunto de valores	`[1, 2, 3]`
`object`	Objeto	`new Persona()`
`null`	Ausencia de valor	`null`

---

7. String

Un `string` es una cadena de caracteres. Se utiliza para almacenar texto.

Ejemplos:

```
$nombre = "Carlos";
$ciudad = "Barcelona";
$mensaje = "Bienvenido a nuestra página";
```

Podemos utilizar comillas simples:

```
$nombre = 'Carlos';
```

o comillas dobles:

```
$nombre = "Carlos";
```

Las comillas dobles son especialmente útiles cuando queremos introducir variables dentro de una cadena:

```
$nombre = "Carlos";

echo "Hola $nombre";
```

Resultado:

```
Hola Carlos
```

También podemos concatenar cadenas mediante `.`:

```
$nombre = "Carlos";
$apellido = "García";

echo $nombre . " " . $apellido;
```

Resultado:

```
Carlos García
```

---

8. Integer

`int` representa números enteros.

Ejemplos:

```
$edad = 18;
$numeroAlumnos = 30;
$temperatura = -10;
```

También podemos realizar operaciones:

```
$a = 10;
$b = 5;

$suma = $a + $b;

echo $suma;
```

Resultado:

```
15
```

---

9. Float

`float` representa números con decimales.

Ejemplos:

```
$precio = 19.95;
$altura = 1.75;
$media = 8.5;
```

Los números decimales utilizan un punto:

```
$numero = 10.5;
```

y no una coma:

```
$numero = 10,5;
```

---

10. Boolean

El tipo `bool` solo tiene dos posibles valores:

```
true
false
```

Ejemplo:

```
$usuarioRegistrado = true;
$usuarioBloqueado = false;
```

Los booleanos se utilizan frecuentemente en condiciones:

```
$usuarioRegistrado = true;

if ($usuarioRegistrado) {
    echo "Usuario registrado";
}
```

---

11. Array

Un `array` permite almacenar varios valores.

Ejemplo:

```
$colores = ["rojo", "verde", "azul"];
```

Podemos acceder a los valores mediante sus índices:

```
echo $colores[0];
```

Resultado:

```
rojo
```

Los índices comienzan normalmente en `0`.

Por tanto:

```
$colores[0] → rojo
$colores[1] → verde
$colores[2] → azul
```

11.1. Arrays asociativos

También podemos utilizar nombres como índices:

```
$persona = [
    "nombre" => "Ana",
    "edad" => 25,
    "ciudad" => "Madrid"
];
```

Para obtener el nombre:

```
echo $persona["nombre"];
```

Resultado:

```
Ana
```

---

12. Null

`null` representa la ausencia de un valor.

Ejemplo:

```
$telefono = null;
```

Posteriormente podemos asignarle un valor:

```
$telefono = "600123456";
```

`null` es útil cuando queremos indicar que una variable todavía no tiene información.

---

13. Variables en PHP

Una variable permite almacenar información que puede cambiar durante la ejecución del programa.

En PHP, las variables comienzan con `$`.

Ejemplo:

```
$nombre = "Juan";
$edad = 20;
$precio = 15.50;
```

Cada variable almacena un valor.

Podemos modificarlo:

```
$edad = 20;

$edad = 21;

echo $edad;
```

Resultado:

```
21
```

---

14. Reglas para nombrar variables

Los nombres de variables deben respetar determinadas reglas.

Ejemplos válidos:

```
$nombre
$edad
$precioProducto
$numero_alumnos
```

Ejemplos no válidos:

```
$1nombre
$nombre-apellido
```

Una variable debe comenzar con `$` y, después, el nombre debe comenzar por una letra o `_`.

Es recomendable utilizar nombres descriptivos:

```
$nombreUsuario = "Ana";
$precioProducto = 50;
$numeroAlumnos = 25;
```

Esto facilita la lectura del programa.

---

15. Variables y tipos dinámicos

PHP es un lenguaje de **tipado dinámico**. Esto significa que una variable puede contener diferentes tipos de valores durante la ejecución.

Por ejemplo:

```
$valor = 100;
```

En este momento es un entero.

Posteriormente podemos hacer:

```
$valor = "Hola";
```

Ahora contiene una cadena.

Aunque PHP permite esto, en proyectos grandes es recomendable escribir código claro y utilizar tipos explícitos cuando corresponda.

---

16. Constantes

Una constante representa un valor que está pensado para permanecer fijo durante la ejecución del programa.

Por ejemplo:

```
const IVA = 0.21;
```

Después podemos utilizarla:

```
echo IVA;
```

Una diferencia importante es que las constantes no utilizan `$`.

16.1. Constantes con `const`

Ejemplo:

```
const NOMBRE_EMPRESA = "TechWeb";
const IVA = 0.21;

echo NOMBRE_EMPRESA;
echo IVA;
```

Es habitual escribir los nombres de las constantes en mayúsculas para distinguirlas visualmente.

16.2. Constantes con `define()`

Otra posibilidad es:

```
define("PI", 3.14159);

echo PI;
```

También:

```
define("VERSION", "1.0");

echo VERSION;
```

---

17. Diferencia entre variables y constantes

Característica	Variable	Constante
Símbolo `$`	Sí	No
Valor modificable	Sí	No se pretende modificar
Ejemplo	`$edad`	`IVA`
Uso	Datos que cambian	Datos fijos

Ejemplo:

```
$edad = 20;
$edad = 21;
```

La variable cambia.

Mientras que:

```
const IVA = 0.21;
```

representa un valor fijo.

---

18. Operadores

Los operadores son símbolos que permiten realizar operaciones sobre valores y variables.

Los principales son:

- Aritméticos.
- Asignación.
- Comparación.
- Lógicos.
- Incremento y decremento.
- Concatenación.
- Ternario.
- Fusión de null.

---

19. Operadores aritméticos

Se utilizan para realizar operaciones matemáticas.

Operador	Operación
`+`	Suma
`-`	Resta
`*`	Multiplicación
`/`	División
`%`	Resto
`**`	Potencia

Ejemplo:

```
$a = 10;
$b = 3;

echo $a + $b;
echo $a - $b;
echo $a * $b;
echo $a / $b;
echo $a % $b;
echo $a ** $b;
```

Si queremos guardar los resultados:

```
$suma = $a + $b;
$resta = $a - $b;
$multiplicacion = $a * $b;
$division = $a / $b;
```

---

20. Operadores de asignación

El operador principal es:

```
=
```

Ejemplo:

```
$numero = 10;
```

También existen operadores abreviados:

```
$numero += 5;
$numero -= 2;
$numero *= 3;
$numero /= 2;
```

Por ejemplo:

```
$numero = 10;
$numero += 5;
```

Es equivalente a:

```
$numero = $numero + 5;
```

Resultado:

```
15
```

---

21. Operadores de comparación

Permiten comparar valores.

Operador	Significado
==	Igual en valor
===	Igual en valor y tipo
!=	Diferente
!==	No idéntico
<	Menor
>	Mayor
<=	Menor o igual
>=	Mayor o igual

Ejemplo:

```
$edad = 20;

if ($edad >= 18) {
    echo "Es mayor de edad";
}
```

21.1. == y ===

Esta diferencia es muy importante.

== realiza una comparación no estricta.

```
5 == "5"
```

El resultado es verdadero porque PHP puede realizar una conversión de tipos para comparar los valores.

=== realiza una comparación estricta:

```
5 === "5"
```

El resultado es falso porque:

```
5    → int
"5"  → string
```

Por tanto, no tienen el mismo tipo.

Cuando necesitamos comprobar tanto el valor como el tipo, === es la opción adecuada.

---

22. Operadores lógicos

Los operadores lógicos permiten combinar condiciones.

AND &&

Las dos condiciones deben ser verdaderas:

```
$edad = 20;
$tieneEntrada = true;

if ($edad >= 18 && $tieneEntrada) {
    echo "Puede entrar";
}
```

OR ||

Al menos una condición debe ser verdadera:

```
$esAlumno = true;
$esProfesor = false;

if ($esAlumno || $esProfesor) {
```

```
    echo "Puede acceder";
}
```

NOT !

Invierte el resultado:

```
$activo = false;

if (!$activo) {
    echo "El usuario está inactivo";
}
```

---

23. Incremento y decremento

Los operadores ++ y -- permiten aumentar o disminuir una variable en una unidad.

Ejemplo:

```
$numero = 5;

$numero++;
```

Resultado:

```
6
```

Para disminuir:

```
$numero--;
```

Resultado:

```
5
```

También podemos utilizar:

```
++$numero;
--$numero;
```

La posición del operador puede ser importante dentro de expresiones.

---

24. Concatenación de cadenas

El operador . permite unir cadenas.

Ejemplo:

```
$nombre = "Ana";
$apellido = "Martínez";

$nombreCompleto = $nombre . " " . $apellido;

echo $nombreCompleto;
```

Resultado:

```
Ana Martínez
```

También existe `.=`:

```
$mensaje = "Hola";

$mensaje .= " Ana";

echo $mensaje;
```

Resultado:

```
Hola Ana
```

---

25. Operador ternario

El operador ternario permite realizar una condición sencilla.

Su estructura es:

```
condición ? valor1 : valor2;
```

Ejemplo:

```
$edad = 20;

$mensaje = ($edad >= 18)
    ? "Mayor de edad"
    : "Menor de edad";

echo $mensaje;
```

Resultado:

```
Mayor de edad
```

Es equivalente a utilizar una estructura `if/else` sencilla.

---

26. Operador de fusión de null

El operador `??` permite proporcionar un valor alternativo cuando una variable es `null` o no está definida.

Ejemplo:

```
$nombre = null;

$nombreMostrar = $nombre ?? "Usuario desconocido";

echo $nombreMostrar;
```

Resultado:

```
Usuario desconocido
```

Es especialmente útil cuando trabajamos con datos opcionales.

---

27. Conversión de datos

La **conversión de datos** consiste en cambiar un valor de un tipo de dato a otro.

Por ejemplo, podemos recibir un número como texto:

```
$edad = "20";
```

y convertirlo en entero:

```
$edad = (int) $edad;
```

Ahora `$edad` es un entero.

La conversión es especialmente importante cuando trabajamos con datos procedentes de formularios, archivos, APIs o bases de datos.

---

28. Conversión mediante casting

PHP permite realizar conversiones utilizando diferentes tipos.

Convertir a entero

```
$numero = "25";

$resultado = (int) $numero;
```

Convertir a decimal

```
$numero = "15.50";

$resultado = (float) $numero;
```

Convertir a string

```
$numero = 100;

$resultado = (string) $numero;
```

Convertir a boolean

```
$numero = 1;

$resultado = (bool) $numero;
```

---

29. Funciones de conversión

PHP también ofrece funciones específicas.

`intval()`

Convierte un valor a entero:

```
$edad = "25";

$edad = intval($edad);
```

`floatval()`

Convierte un valor a `float`:

```
$precio = "19.95";

$precio = floatval($precio);
```

`strval()`

Convierte un valor a string:

```
$numero = 100;

$texto = strval($numero);
```

`boolval()`

Convierte un valor a booleano:

```
$valor = 1;

$resultado = boolval($valor);
```

30. Conversión automática de tipos

PHP puede realizar conversiones automáticas dependiendo del contexto.

Por ejemplo:

```
$numero = "10";
$resultado = $numero + 5;

echo $resultado;
```

En una operación aritmética PHP puede interpretar `"10"` como un número.

Sin embargo, depender excesivamente de conversiones automáticas puede provocar errores o resultados inesperados. Por eso es importante conocer los tipos de datos y realizar conversiones explícitas cuando sea necesario.

---

31. Comprobar el tipo de datos

Podemos utilizar `var_dump()`:

```
$edad = 25;

var_dump($edad);
```

Resultado:

```
int(25)
```

También existen funciones para comprobar tipos:

```
is_string()
is_int()
is_float()
is_bool()
is_array()
is_object()
is_null()
```

Ejemplo:

```
$edad = 20;

if (is_int($edad)) {
    echo "La edad es un número entero";
}
```

---

32. Include

`include` permite incorporar el contenido de un archivo dentro de otro archivo PHP.

Esto permite **reutilizar código** y organizar mejor los proyectos.

Por ejemplo, podemos tener:

```
proyecto/
│
├── index.php
├── menu.php
└── footer.php
```

El archivo `menu.php` puede contener:

```
<nav>
    <a href="index.php">Inicio</a>
    <a href="contacto.php">Contacto</a>
</nav>
```

En `index.php` podemos escribir:

```
<?php

include "menu.php";

echo "<h1>Página principal</h1>";
```

Cuando PHP procesa `index.php`, incorpora el contenido de `menu.php`.

---

33. ¿Para qué sirve include?

`include` es muy útil para evitar repetir código.

Por ejemplo, si tenemos diez páginas y todas tienen el mismo menú, no necesitamos copiar el menú diez veces.

Podemos crear:

```
menu.php
```

y posteriormente incluirlo:

```
include "menu.php";
```

De esta forma, si queremos modificar el menú, solo tenemos que cambiar un archivo.

También podemos utilizar `include` para:

- Cabeceras.
- Menús.
- Pies de página.
- Funciones.
- Configuraciones.
- Componentes reutilizables.

---

34. Require

`require` también permite incluir archivos.

Ejemplo:

```
<?php

require "config.php";

echo "Página cargada";
```

La diferencia más importante entre `include` y `require` aparece cuando el archivo no puede ser encontrado o cargado.

Con `include`, PHP genera una advertencia y, normalmente, continúa la ejecución del programa.

Con `require`, el fallo al cargar el archivo provoca un error que detiene la ejecución del script.

---

35. Diferencia entre include y require

include	require
Incluye un archivo	Incluye un archivo
Si falla, normalmente el script continúa	Si falla, la ejecución se detiene
Adecuado para contenido que no sea imprescindible	Adecuado para archivos necesarios
Ejemplo: `include "footer.php";`	Ejemplo: `require "config.php";`

Por ejemplo, podemos utilizar:

```
include "footer.php";
```

para un pie de página.

En cambio, si tenemos un archivo que contiene configuraciones indispensables:

```
require "config.php";
```

puede ser más apropiado.

---

36. Include_once y require_once

PHP también proporciona:

```
include_once
```

y:

```
require_once
```

Estas instrucciones evitan que un archivo sea incluido más de una vez durante la misma ejecución.

Por ejemplo:

```
require_once "funciones.php";
```

Si `funciones.php` ya se ha incluido, PHP no lo vuelve a incluir.

Esto es especialmente útil cuando tenemos archivos que contienen:

- Funciones.
- Clases.
- Configuraciones.
- Definiciones.
- Conexiones.

---

37. Diferencia entre include_once y require_once

La diferencia respecto a `include` y `require` es principalmente el comportamiento ante múltiples inclusiones.

Instrucción	Si el archivo ya fue incluido	Si el archivo no existe
`include`	Puede incluirlo nuevamente	Warning y normalmente continúa
`include_once`	Evita incluirlo nuevamente	Warning y normalmente continúa
`require`	Puede incluirlo nuevamente	Error que detiene la ejecución

Instrucción	Si el archivo ya fue incluido	Si el archivo no existe
`require_once`	Evita incluirlo nuevamente	Error que detiene la ejecución

---

38. Ejemplo de organización de un proyecto PHP

Una aplicación sencilla podría organizarse de la siguiente manera:

```
mi-proyecto/
│
├── index.php
├── config.php
├── funciones.php
├── header.php
├── menu.php
└── footer.php
```

config.php

```php
<?php

const NOMBRE_EMPRESA = "Mi tienda";
const IVA = 0.21;
```

funciones.php

```php
<?php

function calcularTotal(float $precio, int $cantidad): float
{
    $subtotal = $precio * $cantidad;
    $iva = $subtotal * IVA;

    return $subtotal + $iva;
}
```

header.php

```php
<!DOCTYPE html>
<html>
<head>
    <title>Mi tienda</title>
</head>
<body>
```

menu.php

```php
<nav>
    <a href="index.php">Inicio</a>
    <a href="productos.php">Productos</a>
    <a href="contacto.php">Contacto</a>
</nav>
```

footer.php

```
<footer>
    <p>© 2026 Mi tienda</p>
</footer>

</body>
</html>
```

index.php

```
<?php

require_once "config.php";
require_once "funciones.php";

include "header.php";
include "menu.php";

$producto = "Ordenador";
$precio = "800.50";
$cantidad = 2;

$precio = (float) $precio;

$total = calcularTotal($precio, $cantidad);

echo "<h1>" . NOMBRE_EMPRESA . "</h1>";
echo "<p>Producto: " . $producto . "</p>";
echo "<p>Cantidad: " . $cantidad . "</p>";
echo "<p>Total: " . $total . " €</p>";

include "footer.php";
```

Este ejemplo demuestra cómo se pueden separar las diferentes partes de una aplicación.

---

39. Ejemplo práctico: cálculo de una compra

Podemos crear un pequeño programa que calcule el precio final de una compra.

```
<?php

const IVA = 0.21;

$producto = "Teclado";
$precio = "50.00";
$cantidad = 2;

// Conversión de string a float
$precio = (float) $precio;

// Cálculo del subtotal
$subtotal = $precio * $cantidad;
```

```
// Cálculo del IVA
$impuesto = $subtotal * IVA;

// Precio final
$total = $subtotal + $impuesto;

echo "Producto: " . $producto . "<br>";
echo "Precio: " . $precio . " €<br>";
echo "Cantidad: " . $cantidad . "<br>";
echo "Subtotal: " . $subtotal . " €<br>";
echo "IVA: " . $impuesto . " €<br>";
echo "Total: " . $total . " €";
```

En este programa podemos identificar:

Variable string:

```
$producto = "Teclado";
```

Variable string que posteriormente convertimos a float:

```
$precio = "50.00";
$precio = (float) $precio;
```

Variable integer:

```
$cantidad = 2;
```

Constante:

```
const IVA = 0.21;
```

Operadores aritméticos:

```
*
+
```

Operador de concatenación:

```
.
```

---

40. Ventajas de utilizar include y require

La utilización de archivos externos presenta varias ventajas.

1. Reutilización

Podemos utilizar el mismo código en diferentes páginas.

2. Organización

El proyecto queda dividido en archivos más pequeños.

3. Mantenimiento

Si queremos modificar un menú, podemos cambiar únicamente `menu.php`.

4. Evitar duplicación

No necesitamos copiar el mismo código en todas las páginas.

5. Separación de responsabilidades

Podemos tener un archivo para la configuración, otro para las funciones y otros para la interfaz.

Esto hace que el proyecto sea más fácil de mantener y ampliar.

---

41. Ejercicios prácticos

Ejercicio 1: Variables

Crea un programa que almacene:

- Nombre.
- Edad.
- Ciudad.
- Altura.

Después muestra todos los datos.

Ejemplo de solución:

```
<?php

$nombre = "Carlos";
$edad = 22;
$ciudad = "Barcelona";
$altura = 1.75;

echo "Nombre: " . $nombre . "<br>";
echo "Edad: " . $edad . "<br>";
echo "Ciudad: " . $ciudad . "<br>";
echo "Altura: " . $altura . " metros";
```

Ejercicio 2: Operadores

Calcula el área de un rectángulo.

```php
<?php

$base = 10;
$altura = 5;

$area = $base * $altura;

echo "El área es: " . $area;
```

Resultado:

```
El área es: 50
```

Ejercicio 3: Conversión

Convierte un número almacenado como texto en un número entero.

```php
<?php

$numero = "100";

$numero = (int) $numero;

var_dump($numero);
```

Ejercicio 4: Constantes

Crea una constante que represente el IVA y calcula el precio final de un producto.

```php
<?php

const IVA = 0.21;

$precio = 100;

$impuesto = $precio * IVA;
$total = $precio + $impuesto;

echo "Precio final: " . $total . " €";
```

Ejercicio 5: Include

Crea dos archivos:

```
index.php
mensaje.php
```

En `mensaje.php` escribe:

```php
<?php
```

```
echo "Este mensaje viene de otro archivo.";
```

En `index.php` utiliza:

```
<?php

include "mensaje.php";
```

Al ejecutar `index.php`, aparecerá el mensaje contenido en `mensaje.php`.

---

42. Resumen general

Los conceptos principales de este tema son los siguientes:

Sintaxis

La sintaxis define las reglas que debemos seguir para escribir código PHP correctamente. El código se introduce normalmente mediante `<?php` y las instrucciones suelen terminar con `;`.

Tipos de datos

PHP permite trabajar con diferentes tipos:

- `string`: texto.
- `int`: números enteros.
- `float`: números decimales.
- `bool`: verdadero o falso.
- `array`: conjunto de valores.
- `object`: objetos.
- `null`: ausencia de valor.

Variables

Las variables permiten almacenar información que puede cambiar.

```
$nombre = "Ana";
$edad = 20;
```

Constantes

Las constantes representan valores que están destinados a permanecer fijos.

```
const IVA = 0.21;
```

Operadores

Permiten realizar operaciones matemáticas, asignaciones, comparaciones y operaciones lógicas.

Algunos ejemplos:

```
+   -   *   /   %
=   +=  -=  *=  /=
==  === !=  !== <  >  <=  >=
&&  ||  !
++  --
.   .=
??  ?:
```

Conversión de datos

Permite transformar un valor de un tipo a otro:

```
$numero = (int) "25";
$precio = (float) "19.95";
$texto = (string) 100;
$activo = (bool) 1;
```

También existen funciones como:

```
intval()
floatval()
strval()
boolval()
```

Include y require

Permiten incorporar archivos externos.

```
include "menu.php";
require "config.php";
```

También existen:

```
include_once "funciones.php";
require_once "config.php";
```

---

43. Conclusión

PHP es un lenguaje especialmente útil para el desarrollo de aplicaciones web dinámicas. Para comenzar a utilizarlo correctamente es necesario conocer sus fundamentos y comprender cómo se representan y manipulan los datos.

La **sintaxis** establece las reglas necesarias para escribir código válido. Los **tipos de datos** permiten distinguir entre texto, números, valores booleanos, arrays, objetos y valores nulos. Las **variables** permiten almacenar información que puede cambiar

durante la ejecución, mientras que las **constantes** permiten definir valores que deben mantenerse fijos.

Los **operadores** son fundamentales para realizar cálculos, comparar valores, combinar condiciones y manipular cadenas de texto. Por otra parte, la **conversión de datos** permite transformar información entre diferentes tipos, algo especialmente importante cuando trabajamos con datos procedentes de formularios u otras fuentes externas.

Finalmente, `include`, `require`, `include_once` y `require_once` permiten dividir un proyecto en varios archivos y reutilizar código. Esto facilita la organización y el mantenimiento de las aplicaciones.

El dominio de estos conceptos proporciona una base sólida para continuar aprendiendo PHP y avanzar posteriormente hacia temas más complejos, como **funciones, formularios, sesiones, programación orientada a objetos, bases de datos, autenticación y desarrollo de aplicaciones web completas**.